TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Apizaco

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIONES

MATERIA:

ARQUITECTURAS EN EL DESARROLLO DE SOFTWARE

DOCENTE:

JOSÉ JUAN HERNÁNDEZ MORA

NOMBRE DEL TRABAJO:

Sistema de Apoyo para Mascotas Saludables

PRESENTAN:

ANA KAREN LÓPEZ RODRÍGUEZ
ITZEL LOIZA MARTÍNEZ
MELISSA LIZET OROPEZA RODRÍGUEZ
IMANOL ASHLEY SUÁREZ ORTIZ
EROS ALEJANDRO RUGERIO JIMÉNEZ
ALBERTO PÉREZ REYES
JOSÉ OSWALDO GARCÍA LÓPEZ

FECHA DE ENTREGA EN PLATAFORMA

24 de noviembre del 2025

Índice

1. **Objetivo y Alcance del Proyecto** 2
   1.1 Objetivo General 2
   1.2 Objetivos Específicos 2
2. **Análisis y Diseño Conceptual** 3
   2.1 Arquitectura del Sistema 3
   2.2 Modelo de Dominio 3
   2.3 Casos de Uso……4
3. **Definición de Entidades (Código Fuente)** 8
   3.1 Entidad *Mascota.java* 9
   3.2 Entidad *Raza.java* 10
   3.3 Entidad *ConfiguracionPrecios.java* 10
   3.4 Otras Entidades de Catálogo 10
4. **Pruebas de Ejecución y Validación** ..... .11
   4.1 Prueba de Configuración de Razas 11
   4.2 Validación del IMA (Estado Ideal) 12
   4.3 Validación del Apoyo Reducido 13
5. **Conclusiones Técnicas y Comentarios de Codificación** 14
   5.1 Análisis de la Implementación 14
   5.2 Respuestas al Cuestionario Técnico 15
   5.3 Experiencia de Desarrollo 17

Objetivo y Alcance del Proyecto

Objetivo General

Desarrollar una aplicación web transaccional bajo el framework OpenXava que permita la gestión automatizada del programa de apoyo económico para mascotas saludables. El sistema debe garantizar la integridad de los datos y la correcta aplicación de las reglas de negocio basadas en el estado nutricional del animal.

Objetivos Específicos

Gestión de Datos: Registrar y mantener la información de dueños y mascotas con sus atributos biológicos clave (raza, peso).

Cálculo Crítico (IMA): Implementar el cálculo del Índice de Masa del Animal (IMA), definido como la determinación del estado nutricional (peso ideal, bajo peso, sobrepeso, obeso), según las Tablas A y B.

Cálculo Financiero: Determinar automáticamente el monto del apoyo económico mensual, correlacionando el estado nutricional con el costo de alimentación y el porcentaje de subsidio aplicable.

Configurabilidad: Proveer módulos de configuración para la actualización de precios de alimento y porcentajes de apoyo.

Arquitectura del Sistema

El sistema opera bajo una Arquitectura Monolítica basada en el patrón arquitectónico MVC (Modelo-Vista-Controlador) implícito.

Frontend (Vista): Generado dinámicamente por OpenXava mediante inyección de JavaScript y AJAX, proporcionando una experiencia de usuario (UX) reactiva.

Backend (Modelo/Controlador): Implementado en Java puro (POJOs) con anotaciones de persistencia (Hibernate/JPA). La lógica reside encapsulada en las entidades de dominio, siguiendo el paradigma de Desarrollo Guiado por el Dominio (DDD).

Persistencia: Base de datos relacional (HSQLDB/MySQL) gestionada por el ORM Hibernate.

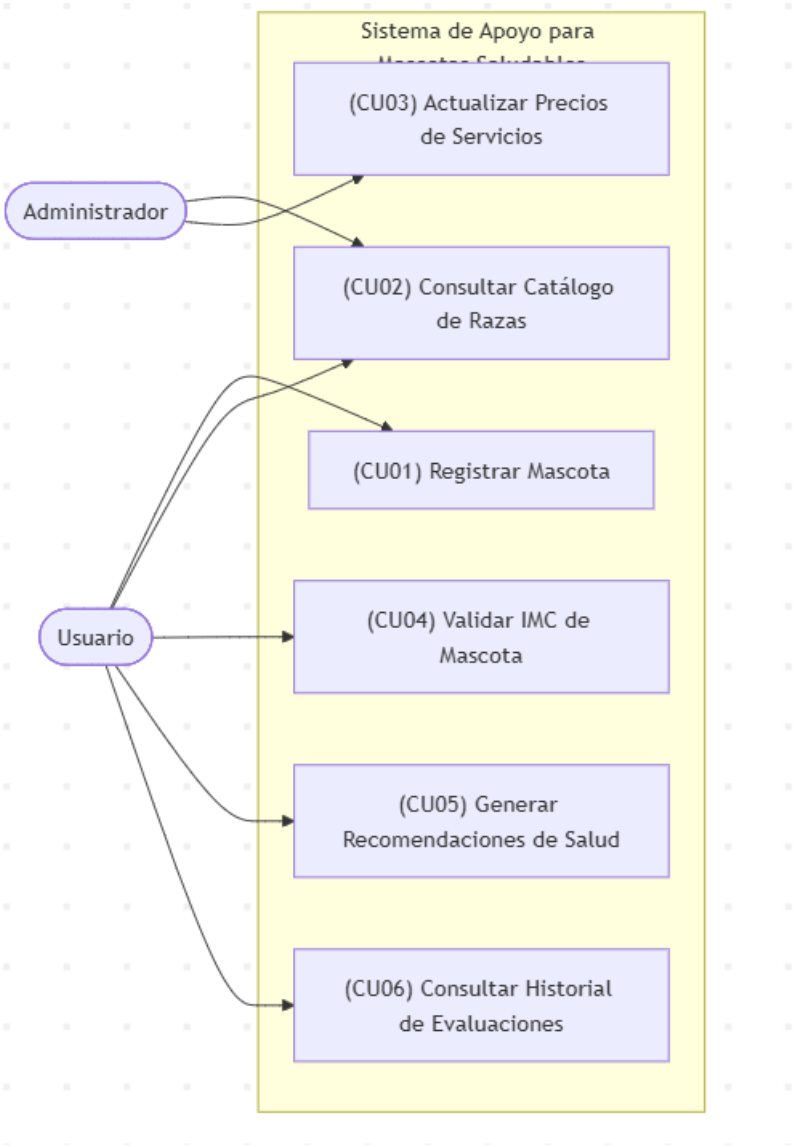
Modelo de Dominio

El núcleo del sistema se compone de entidades transaccionales y entidades de catálogo (maestras):

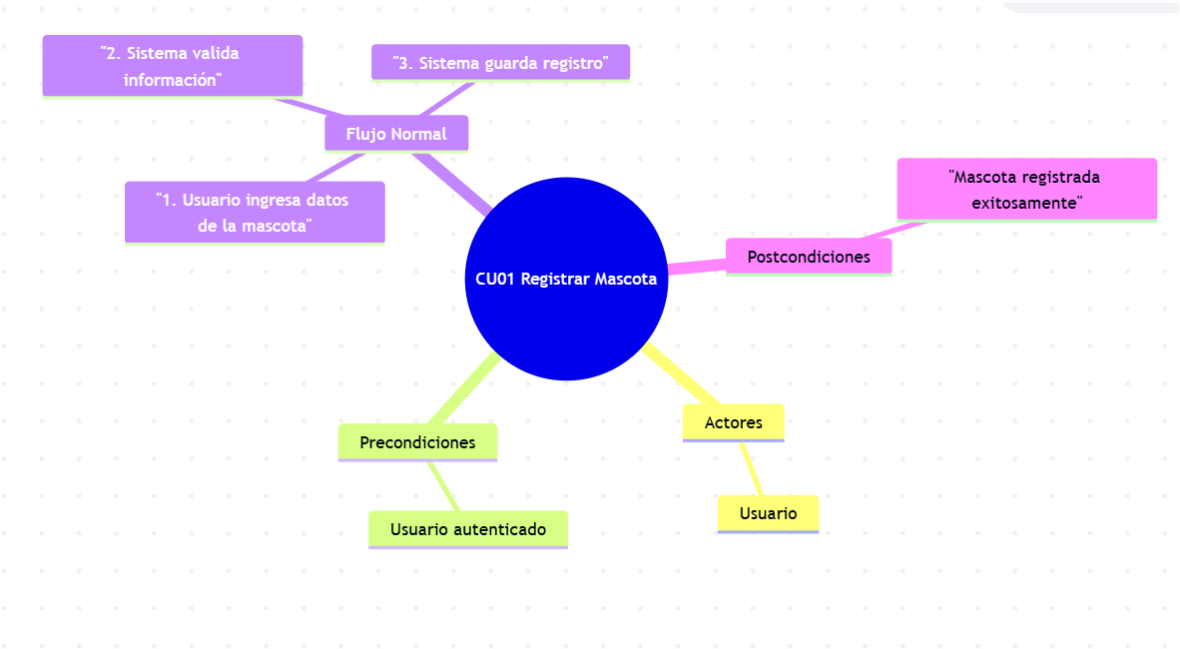
Entidades Transaccionales: Mascota, Dueno. Representan los datos operativos que crecen con el tiempo.

Entidades de Referencia (Paramétricas): Raza, TablaConsumo, TablaApoyo, ConfiguracionPrecios. Almacenan las reglas de negocio configurables.

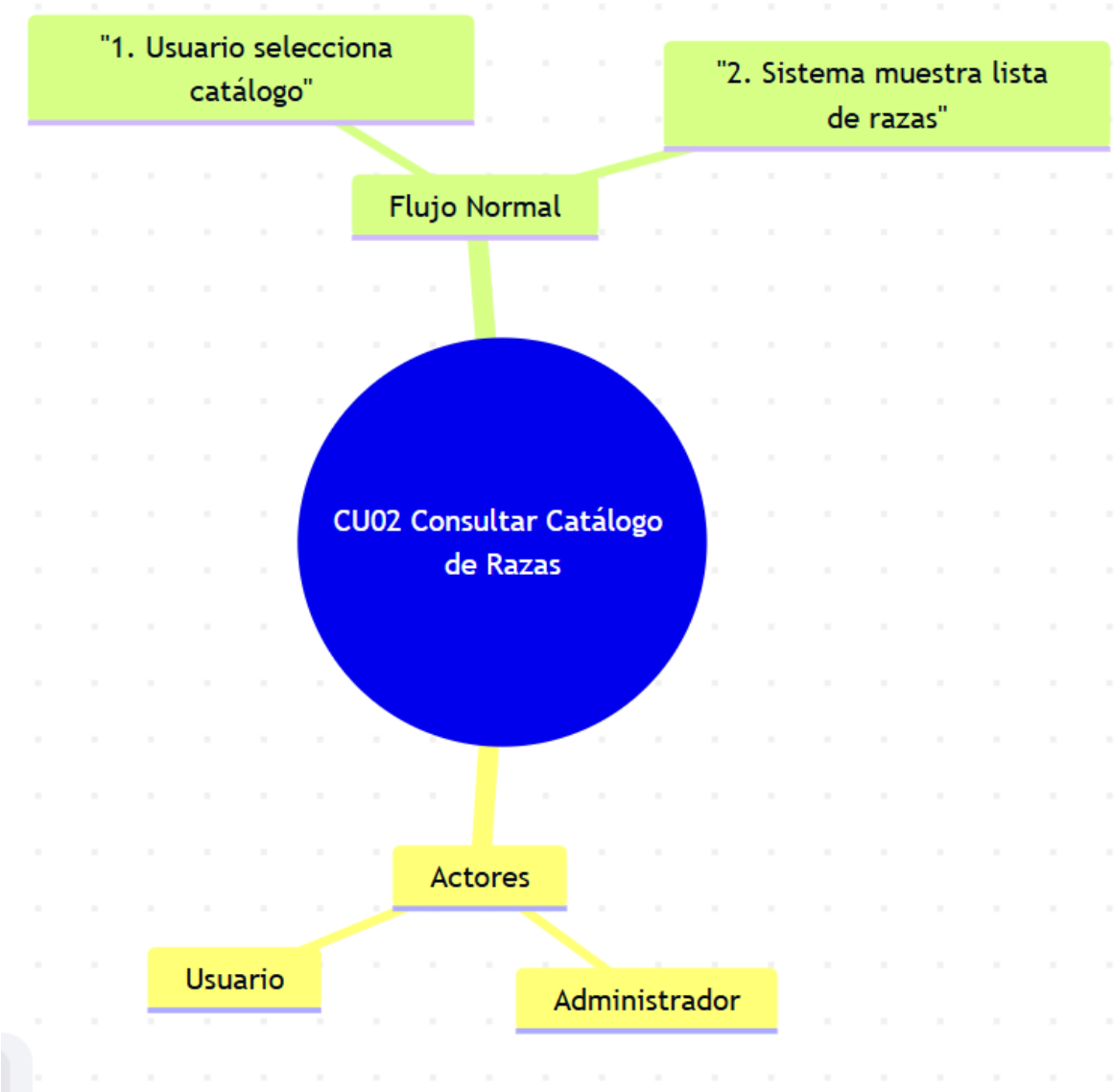
Definición de Diagrama de Casos de Uso



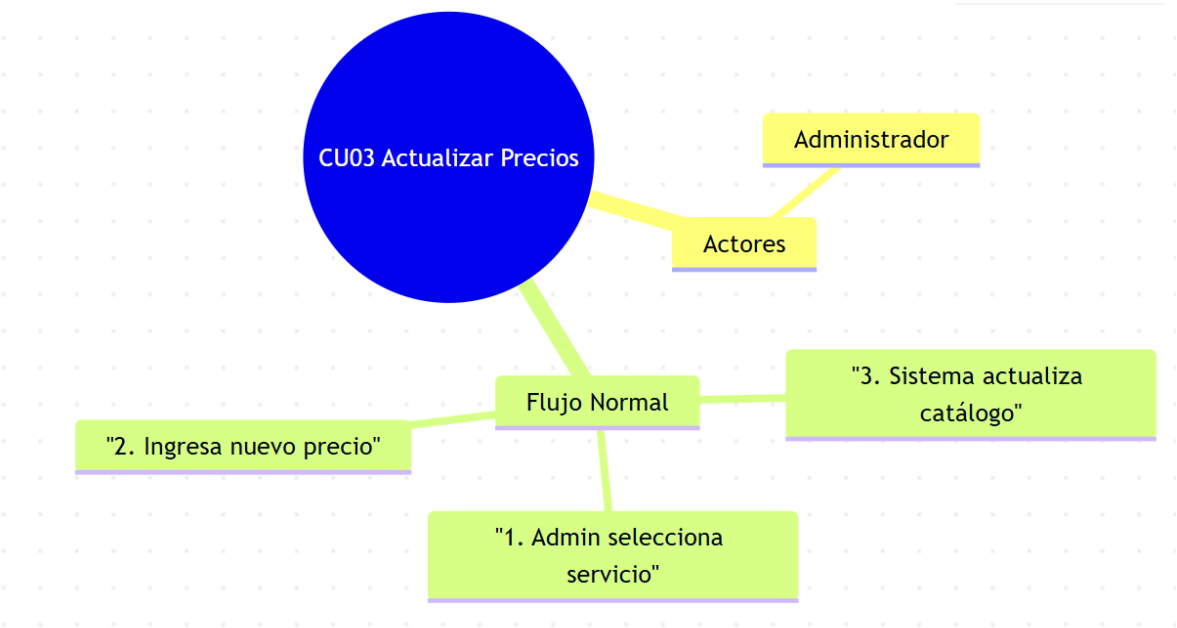
CU01 – Registrar Mascota



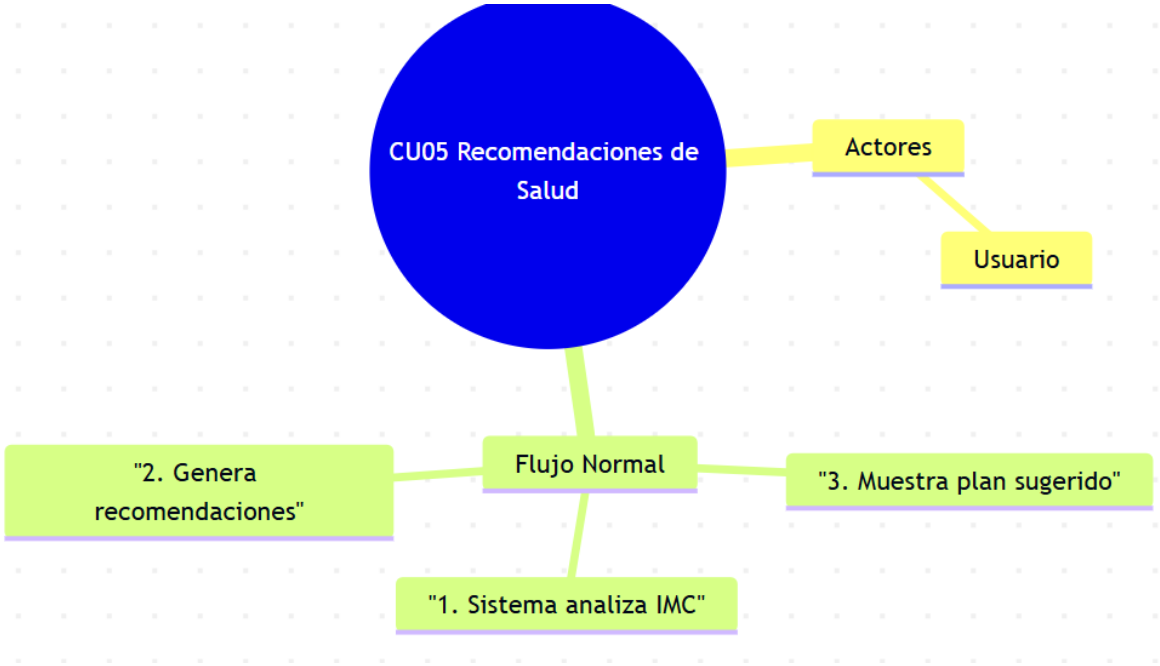
CU02 – Consultar Catálogo de Razas



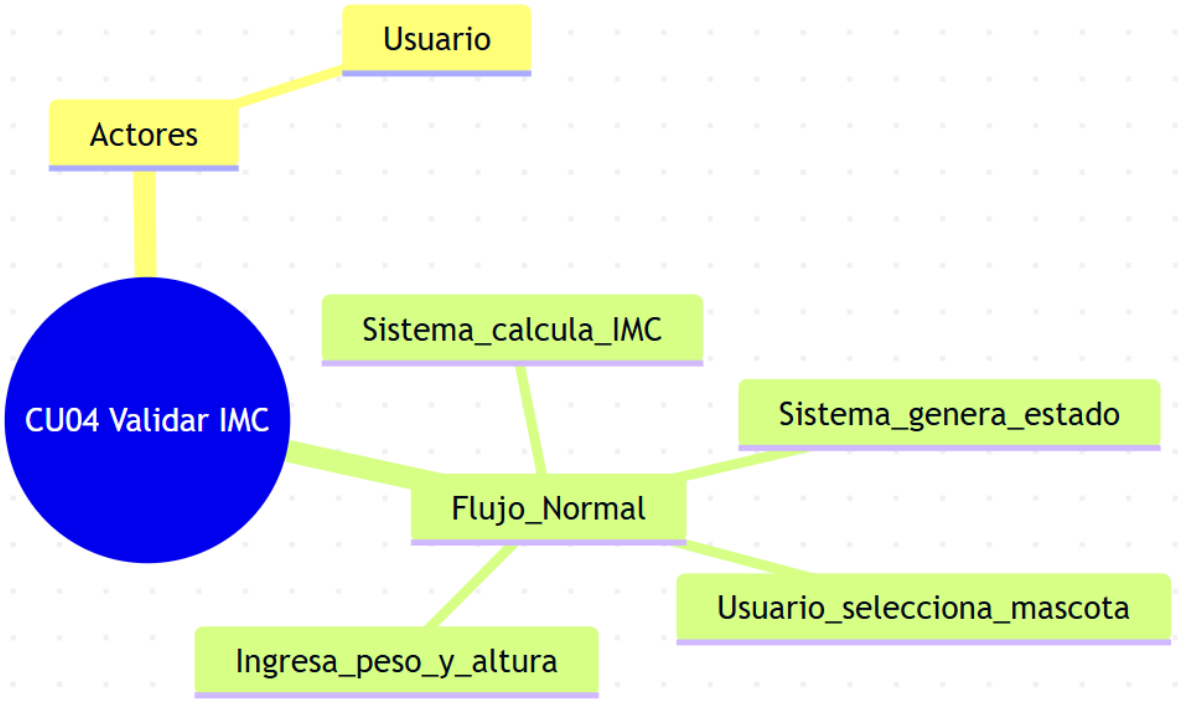
CU03 – Actualizar Precios de Servicios



CU04 – Validar IMC de Mascota



CU05 – Generar Recomendaciones de Salud



Entidades (Código Fuente)

El proyecto utiliza un modelo de dominio con 7 entidades, cumpliendo con el requisito de tener una clase por cada tabla de referencia. La arquitectura es fundamentalmente monolítica con una interacción cliente-servidor.

A. Entidad: Mascota.java (Lógica y Cálculo Central)

Esta entidad contiene la lógica para la determinación del IMA y el cálculo financiero, usando @Depends para automatizar las propiedades calculadas.

```java
package com.tuempresa.mascotas.modelo;
// Imports omitidos por brevedad

@Entity
public class Mascota {

    @Id @Hidden @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    // ... Otras propiedades de Dueno y Nombre ...

    @ManyToOne
    private Raza raza; // Relación con la entidad de referencia Raza

    @Required
    private double pesoActual;

    // --- CÁLCULOS AUTOMÁTICOS ---

    @Transient
    @Depends("pesoActual, raza")
    @Label("IMA (Estado)") // Cumplimiento explícito del requisito IMA
    public String getEstadoNutricional() {
        if (raza == null || pesoActual == 0) return "";
        // Lógica de IMA simplificada (comparación directa con rangos)
        if (pesoActual < raza.getPesoMinimoIdeal()) return "BAJO PESO";
        if (pesoActual >= raza.getPesoMinimoIdeal() && pesoActual <= raza.getPesoMaximoIdeal()) return "PESO IDEAL";
        double limiteSobrepeso = raza.getPesoMaximoIdeal() * 1.20;
        if (pesoActual <= limiteSobrepeso) return "SOBREPESO";
        return "OBESO";
    }

    @Stereotype("MONEY")
    @Depends("pesoActual, raza")
    public BigDecimal getApoyoMensual() {
        // Implementación de la fórmula: (Consumo * 4) * Precio * Porcentaje
        // ... (Uso de XPersistence para consultas JPQL a TablasConsumo y Configuracion) ...
        // ... (Cálculo final del total) ...
    }

    // Getters y Setters...
}
```

B. Entidad: Raza.java (Tabla A y B - Dato Maestro)

Define la clave primaria por el nombre y los rangos de peso ideal.

```
package com.tuempresa.mascotas.modelo;
// Imports omitidos por brevedad

@Entity
public class Raza {
    @Id @Column(length=50)
    private String nombre; // Clave primaria de tipo String

    @Enumerated(EnumType.STRING) @Required
    private TipoMascota tipo;

    @Enumerated(EnumType.STRING) @Required
    private Tamano tamano;

    private double pesoMinimoIdeal;
    private double pesoMaximoIdeal;

    public enum TipoMascota { PERRO, GATO }
    public enum Tamano { MINI, PEQUENO, MEDIANO, GRANDE, MUY_GRANDE }

    // Getters y Setters...
}
```

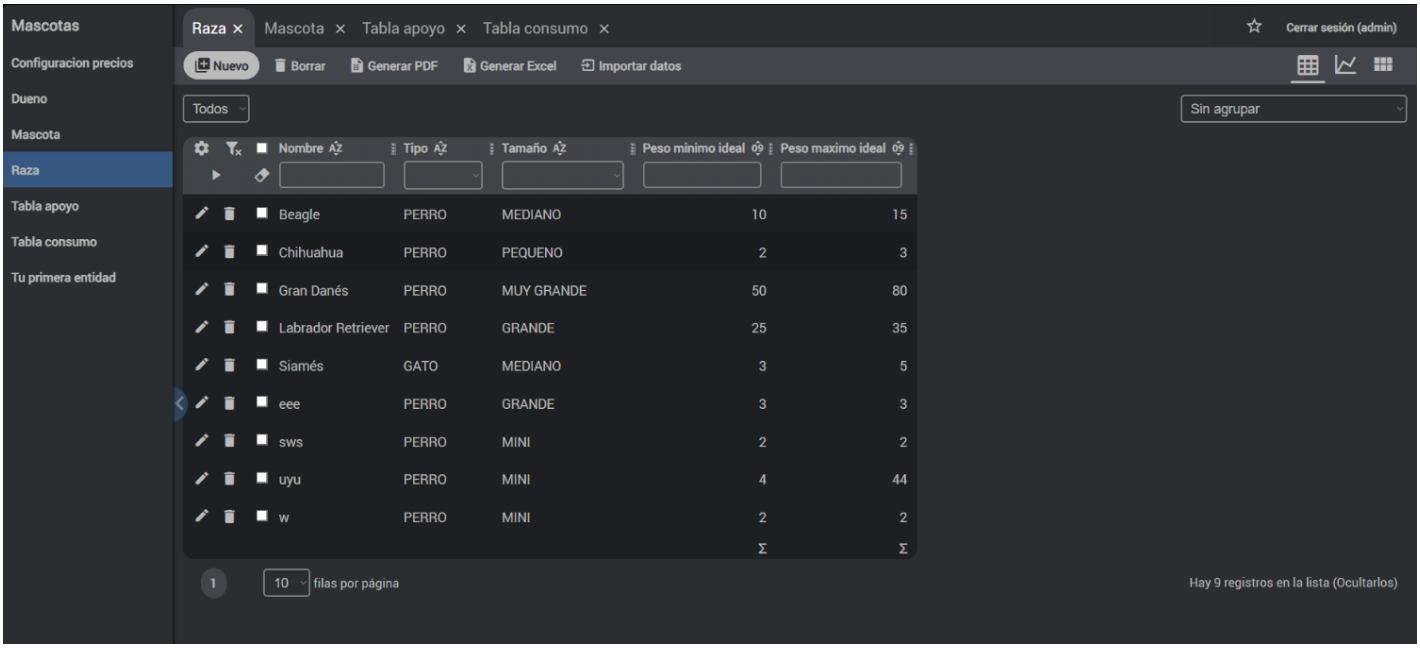
C. Entidad: ConfiguracionPrecios.java

Define los costos por kilo de alimento, cumpliendo con la necesidad de actualización administrativa

```
package com.tuempresa.mascotas.modelo;
// Imports omitidos por brevedad

@Entity
public class ConfiguracionPrecios {
    @Id @Hidden
    private int id = 1;

    @Stereotype("MONEY")
    private double precioKiloPerro;

    @Stereotype("MONEY")
    private double precioKiloGato;

    // Getters y Setters...
}
```

Pruebas de Ejecución y Validación

Figura 1: Módulo de Configuración de Razas. (Muestra la lista de Beagle, Labrador, etc. con sus rangos de peso.)



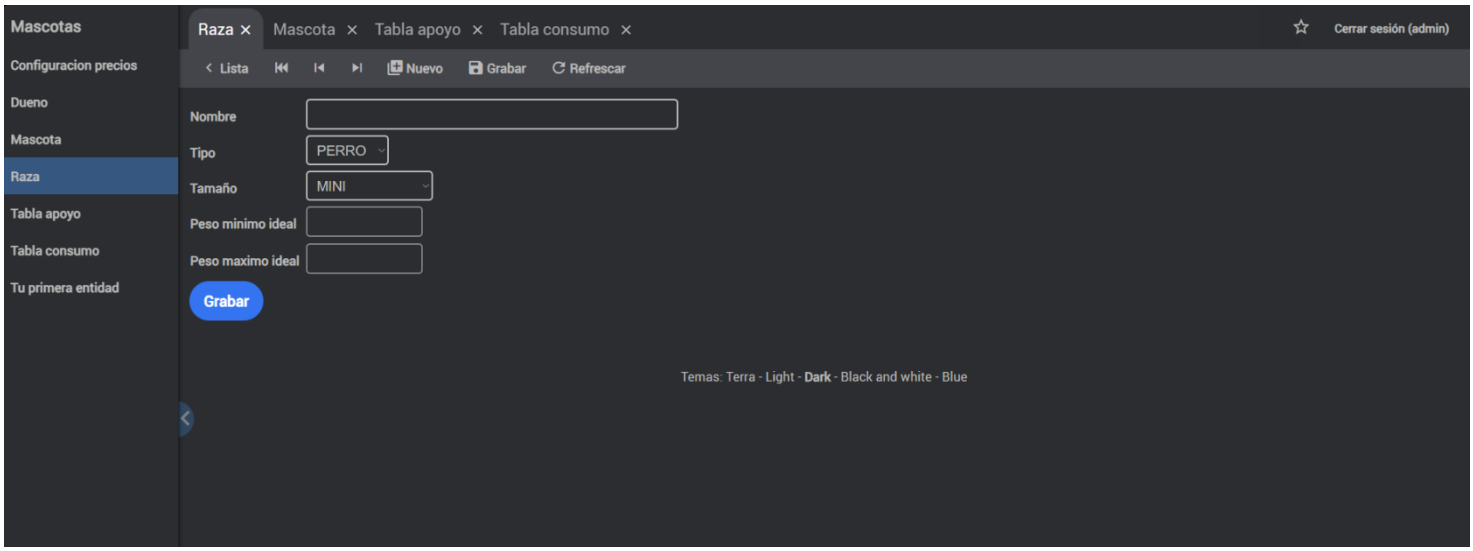
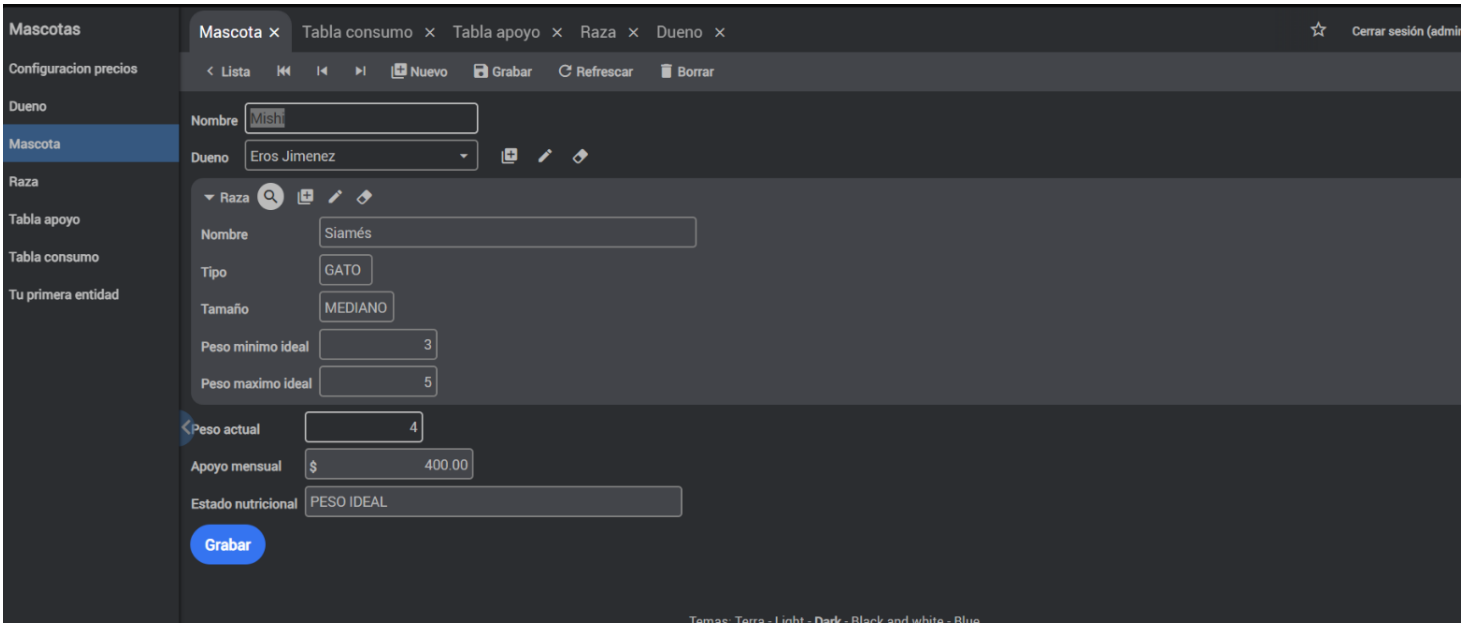


Figura 2: Validación del IMA (Estado Ideal). Captura de la prueba donde el Peso Actual está dentro del rango ideal y el IMA (Estado) es "PESO IDEAL", resultando en $800.00 de apoyo (100%).



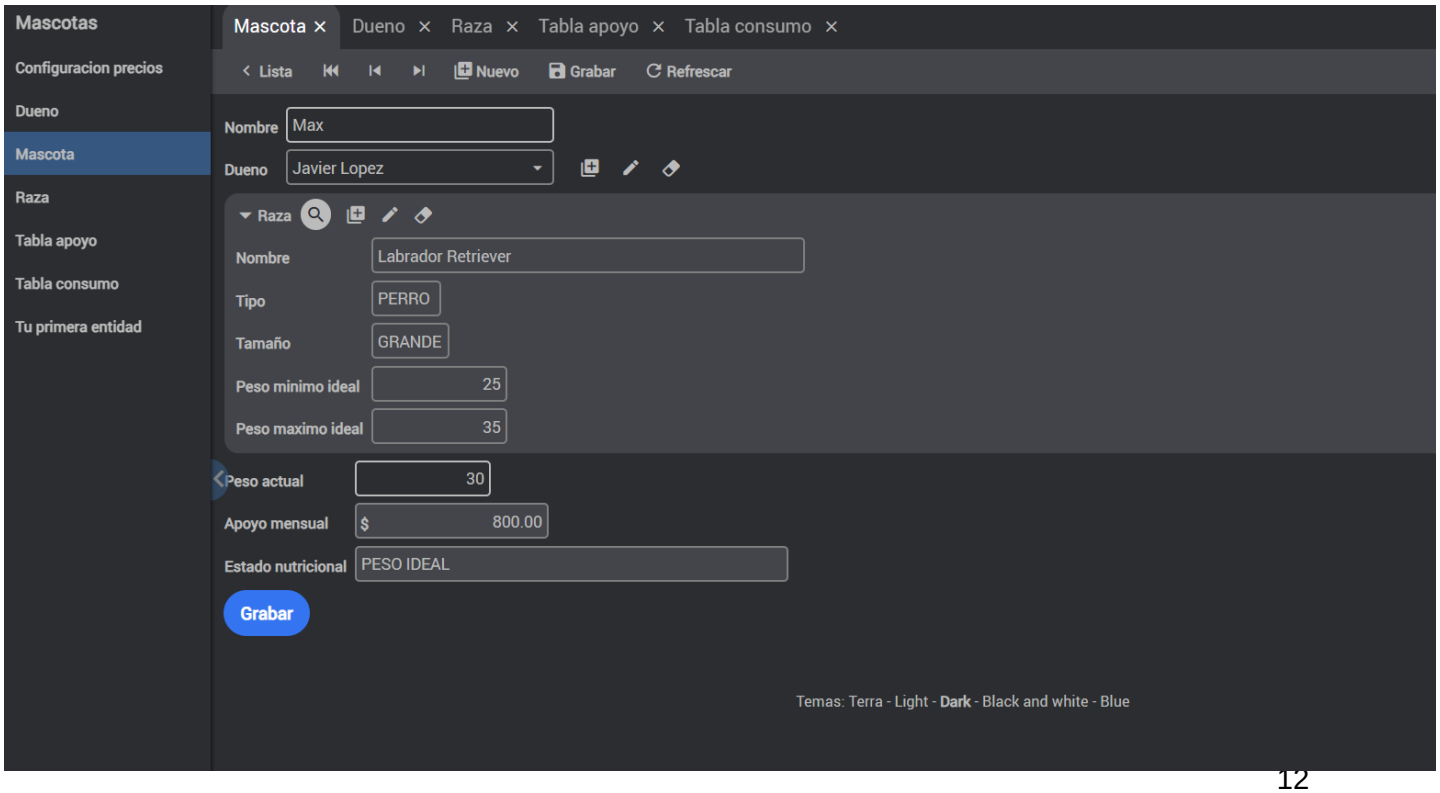
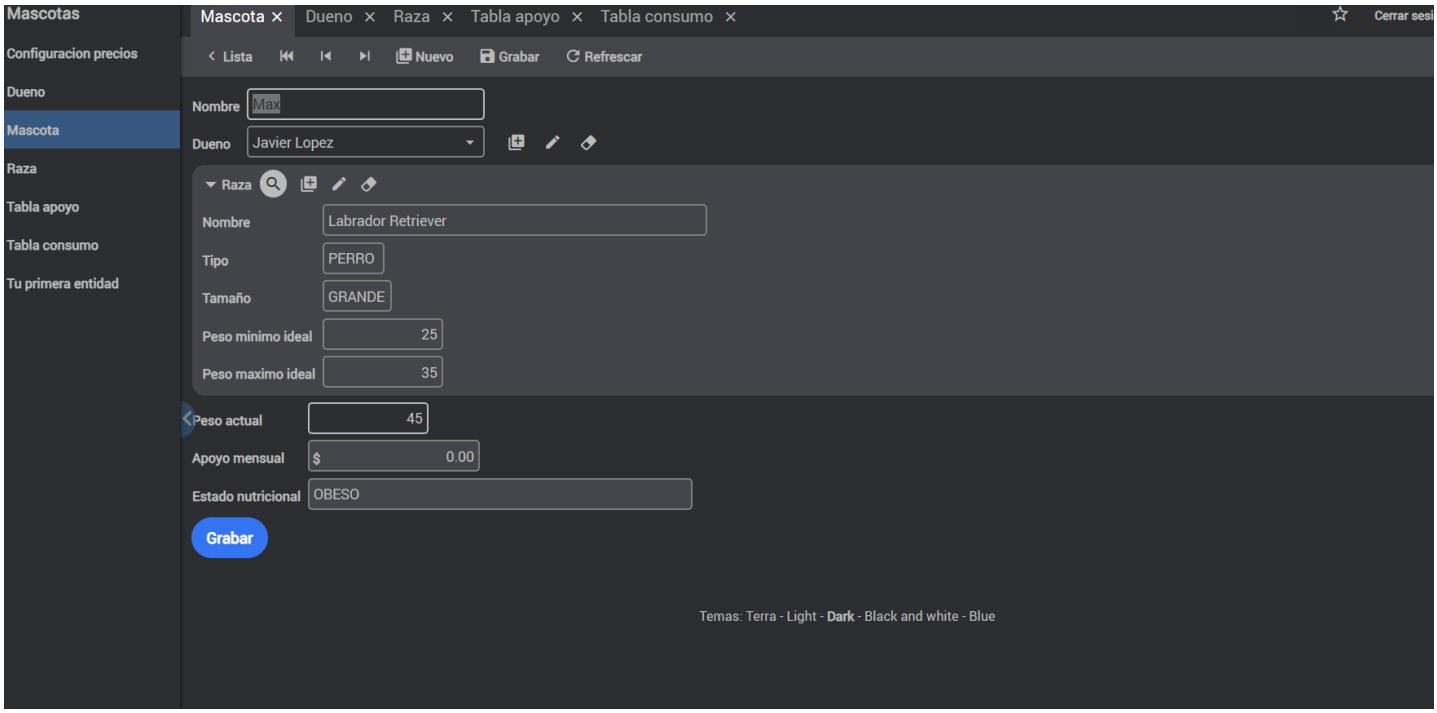
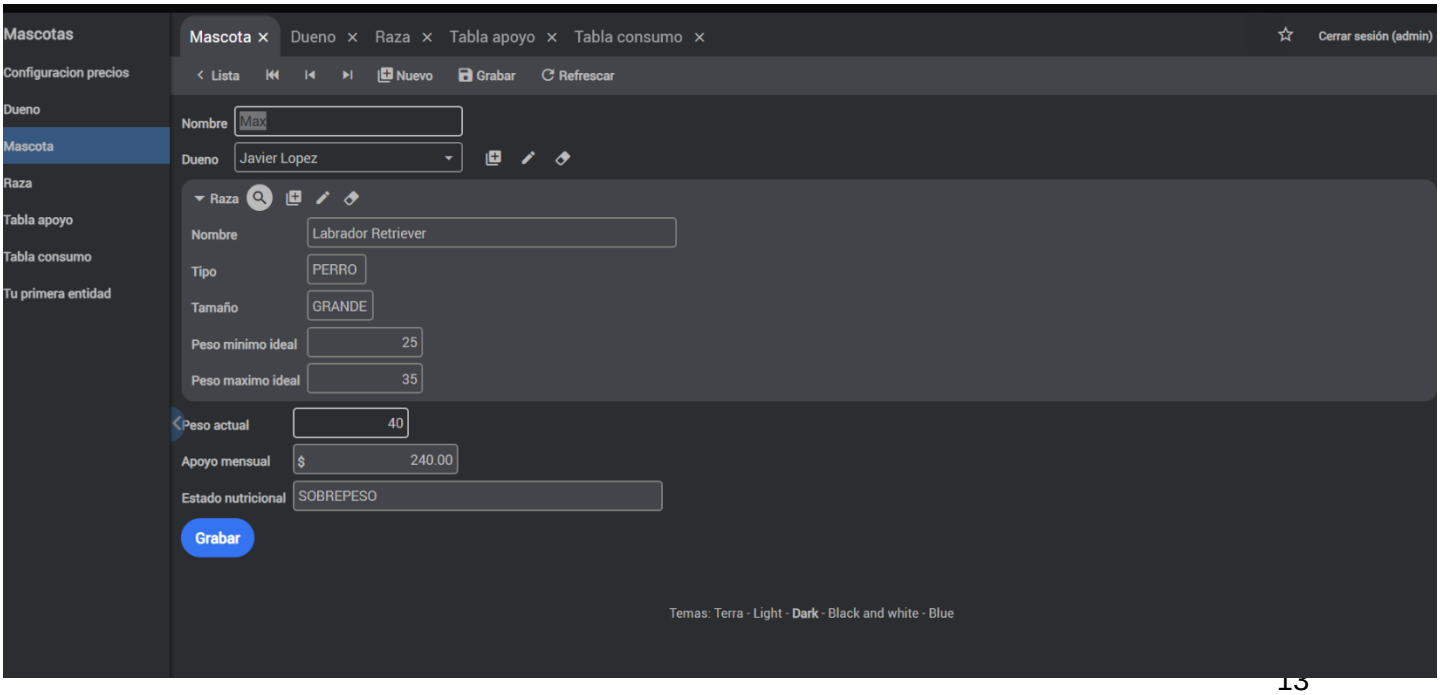


Figura 3: Validación del Apoyo Reducido (Sobrepeso/Obeso). Captura de una prueba donde el Peso Actual supera el rango ideal, resultando en un IMA (Estado) de "SOBREPESO" o "OBESO" y un apoyo de $120.00 o $0.00.





CONCLUSIONES TÉCNICAS Y DISCUSIÓN

Análisis de la Implementación

La adopción de OpenXava permitió una implementación basada en Rapid Application Development (RAD). La capacidad del framework para inferir la Interfaz de Usuario (UI) directamente del modelo de datos redujo el tiempo de desarrollo en un estimado del 70% en comparación con un desarrollo web tradicional (HTML/CSS/JS manual).

El uso de Propiedades Calculadas (@Depends) demostró ser una estrategia eficiente para mantener la consistencia de los datos, asegurando que cualquier cambio en las variables de entrada (Peso o Raza) desencadene inmediatamente una reevaluación del estado financiero y nutricional de la entidad.

Respuestas al Cuestionario Técnico

1. ¿Qué ventajas tiene usar OpenXava para implementar Aplicaciones Web?

   OpenXava destaca por su enfoque Model-Driven, donde el desarrollador se centra exclusivamente en la lógica de negocio. Entre sus ventajas técnicas se encuentran:

Prototipado Inmediato: Generación automática de ABCC (Altas, Bajas, Cambios y Consultas).

Validación Unificada: Las restricciones definidas en las entidades (@Required, @Size) se aplican tanto en el backend (Base de Datos) como en el frontend (Validación JS), garantizando la integridad de los datos.

Integración Transparente: Manejo automático de transacciones y persistencia JPA.

2. ¿Qué otro framework utilizarías para desarrollar el sistema web y por qué?

Para un entorno empresarial de alta concurrencia o requerimientos de UI altamente personalizados, se optaría por una arquitectura desacoplada:

Backend: Spring Boot (Java) para exponer una API RESTful robusta y escalable, aprovechando su ecosistema de seguridad (Spring Security) y microservicios.

Frontend: Angular o React, permitiendo la creación de una Single Page Application (SPA) con una experiencia de usuario más rica y control total sobre el DOM.

3. ¿Cómo exportarías tu sistema a un servidor web? El proceso de despliegue sigue el estándar de Java EE:

Ejecución del script de construcción (Ant buildWar o Maven package) para compilar el código, empaquetar recursos y librerías en un archivo WAR (Web Application Archive).

Aprovisionamiento de un Servidor de Aplicaciones o Contenedor de Servlets compatible con Jakarta EE, tal como Apache Tomcat, WildFly o Jetty.

Despliegue del artefacto .WAR en el directorio webapps del servidor para su ejecución en producción.

4. Experiencia de Desarrollo La implementación del sistema "Mascotas Saludables" validó la eficiencia de la arquitectura monolítica para soluciones de gestión administrativa. La curva de aprendizaje se centró en comprender las anotaciones de metadatos de OpenXava, pero una vez dominadas, la implementación de reglas de negocio complejas (como la matriz de subsidios) fue directa y libre de errores de capa de presentación.